

Understanding the Dynamics of License Selection in OSS

ANONYMOUS AUTHOR(S)

Un- or incorrect-licensed Open Source Software (OSS) code can poison supply chains and inhibit innovation. Many OSS developers are not aware of these implications or make choices that are sub-optimal. We aim to understand the current state of OSS licensing by developing a preliminary theory and model for license choice and testing it using a comprehensive and nearly complete dataset of OSS licenses. Our theory makes four sets of predictions regarding license “stickiness” and choice based on a number of observable project context factors. In contrast to prior work, we find that over 80% of OSS projects have no license. We find that copyleft licenses are most sticky, permissive licenses most common, that social, technical, and ideological factors affect the choice of license. Future research into better understanding reasons for and ways to change the “no license” defaults and devise ways to anticipate license type for a project would have the most practical impact on reducing licensing risks and enable more innovation.

CCS Concepts: • **Software and its engineering** → **Software creation and management**; • **General and reference** → **Empirical studies**.

Additional Key Words and Phrases: Software License, Open Source Software, Open Source License, World of Code

ACM Reference Format:

Anonymous Author(s). 2024. Understanding the Dynamics of License Selection in OSS. 1, 1 (September 2024), 21 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Open Source Software (OSS) has become a cornerstone of modern software development, fostering innovation, knowledge exchange, and the widespread adoption of software tools and libraries. The licensing of OSS projects dictates how software can be used, modified, and distributed. Not all open source software (OSS) participants are aware of or follow the ramifications of licensing [Almeida et al. 2017]. For example, their code is under exclusive copyright by default if no license is specified. The lack of knowledge, and/or, possibly, the lack of enforcement, results in reuse of unlicensed code in contexts where license incompatibilities are likely. For example, with the rise of large language models (LLMs), a significant portion of the massive source code used for training consists of code without any license [Xu et al. 2024; Yu et al. 2023].

Our aim is to better understand the actual practice of license selection, including the choice of no license, specifically because copying no-license code is likely to result in license violation, and because other studies have found that much of the copied code, even in large and popular projects, is copied from small and inactive projects [Jahanshahi et al. 2024a] that tend to have no license. The problem may be propagated further, as the active and popular projects are often upstream dependencies for numerous projects downstream. Hence if such inappropriate use is detected, it can affect a very large part of the OSS ecosystem, not unlike what happened with *leftpad* [Chowdhury et al. 2021]. Prior work found relatively low frequency of repositories with no license (e.g. 21.12% in [Wu et al. 2024] and 10.51% in [Cui et al. 2023]), that, as we demonstrate,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2024/9-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

greatly underestimates the proportion of projects with no license due to heavily filtering based on size, maturity, popularity, and activity.

In OSS, any decision is often made based on perceived supply chain concerns, such as choosing to participate in a project that will become popular, hoping to have a large number of direct or downstream users (and followers), selecting upstream dependencies that appear healthy, responsive, and active [Dabbish et al. 2012; Ma et al. 2020; Malviya-Thakur et al. 2023], and so on. License choice has clear and far reaching effects on the supply chains in terms of attracting and retaining participants, having downstream dependents or users, and, of course on ways the code may be (re-)used. Despite this, we are not aware of any prior studies that model license choice nor any comprehensive theories that would explain it.

Our aim is, therefore, to establish and test a preliminary theory that helps explain license choice in OSS software. The theory could help explain some of the reasons why a large fraction of projects have no license, why some license violations (such as use of unlicensed code) appear to be tolerated, and what are factors that may come into play when a developer (or an organization) chooses a license. Such theory can potentially lead to better tools that help developers choose the most suitable license according to their objectives, increase the proportion of projects with a license (hence spurring innovation that is presently inhibiting actors who do pay attention to licensing terms from using unlicensed code), and, more generally, reduce risks arising from license incompatibilities in OSS supply chains.

To accomplish our goals we start from reviewing literature in areas that may help us formulate a preliminary theory, such as OSS developer motivation, company involvement in OSS, existing theories on how technology and libraries are selected and spread, as well as literature on OSS licensing. We pay particular attention to phenomena that could be measured at scale in order to make our theory easily testable. We then utilize a curated list of OSS projects with identified licenses and license types for nearly the entire OSS ecosystem in order to capture not only the most popular projects but to include small and inactive projects that are less likely to have a license. Based on this large curated list of licences and projects, we use our preliminary theory to operationalize key factors where theory predicts a certain effect on the license choice. We first test if certain types of licenses have longer retention rates based on predictions of our theory. We then employ a multinomial regression model and a stratified sampling approach, to test which of the theory-predicted contextual factors impact license choice.

Our analysis of the types and prevalence of licenses across more than 131 million OSS projects, finds that 83% of these projects lack a formal license. Among the licensed projects, the MIT license was the most commonly used, accounting for 65% of all licensed projects, followed by the Apache 2.0 and ISC licenses. Although permissive licenses dominate overall, copyleft licenses like the GPL exhibited high retention rates, reflecting a strong commitment to software freedom within certain communities. In comparison to prior studies we find four-fold larger prevalence of unlicensed code.

Our theory-derived project characteristics that should influence license selection in OSS projects were tested using a multinomial regression model. As predicted, the number of commits, project age, and file count were found to significantly affect license choice. Projects with more commits and older projects tended to adopt copyleft licenses, whereas permissive licenses were more commonly chosen by newer projects and those with more contributors. Additionally, projects with a larger number of files were more likely to adopt conditional open or weak copyleft licenses, while those with fewer files favored permissive licenses. The choice of programming language also played a critical role, with languages such as JavaScript and Go leaning toward permissive licenses, while C/C++ and Perl showed higher adoption of copyleft licenses.

Our contributions include (a) a preliminary theory of license choice, (b) a comprehensive account of almost all license types, including minor and non-semantic variations, at nearly the full scale

of the OSS ecosystem, (c) theory-derived predictors of license choice, and (d) an evaluation of the preliminary theory using a multinomial model of license choice.

2 Theory Development

To model license choice, several strands of prior research may be relevant. First is the line of work on motivations of individuals [Von Krogh et al. 2012] and businesses [Harhoff and von Hippel 2002] to participate in OSS. Second, technical, social, and other factors that either constraint certain choices in software development or make them more convenient [Ma et al. 2020]. Third, the legal implications and the need to manage compliance with licenses in open source software. Fourth, the specific plans and objectives of the project, and how each license choice may advance and hinder these goals [Sen et al. 2008].

2.1 Motivation

An extensive literature review by Von Krogh et al. [2012] categorizes OSS participant motivations into ideology, altruism, kinship, fun, reputation, reciprocity, learning, own-use, career, and pay. Some of these motivations can influence the type of licenses contributors prefer while others do not have a clear relevance. For instance, ideology and reciprocity are likely to favor strongly copyleft licenses (over all other considerations), which require code sharing, while altruism may lean toward more permissive licenses. Kinship should drive adoption of licenses used by developers' social network, while own-use motivation appears to be orthogonal with license choice and may allow for other factors, like convenience and compatibility to be dominant.

In case of commercial involvement, motivations may diverge from individual contributors, even if they generally align with profit maximization [Harhoff and von Hippel 2002]. Studies of open and gated software communities highlight the difficulties firms face in balancing community-based value creation with private value appropriation [Shah 2006]. However, community sponsorship and licensing strategies can address these tensions. For instance, restrictive licenses can ensure long-term control over contributions, whereas nonmarket¹ sponsors might alleviate concerns about the project's sustainability without the need for restrictive licenses [Stewart et al. 2006]. This highlights how the nature of the sponsor—market or nonmarket—may influence licensing strategies and developer motivations.

Empirical studies further emphasize the connection between license choice and project structure. Fershtman and Gandal [2007] applied economic theories of motivation and found that factors such as status, signaling, or intrinsic motivation for participation in OSS projects are linked to restrictive licenses. They observed that restrictive licenses attract more contributors per project, while permissive licenses result in higher productivity per contributor. Similarly, research into firm involvement in OSS shows that licensing choices may be driven by strategic goals. For instance, firms may adopt permissive licenses to facilitate bundling proprietary and open source code [Lerner and Tirole 2002]. Dahlander and Magnusson [2008] identified three strategies firms use to engage with OSS communities: (1) accessing community development to extend resources, (2) aligning their strategy with community efforts, and (3) assimilating the community to integrate and share outcomes. Different commercial models necessitate distinct licensing strategies. Wagstrom et al. [2010] and Zhou et al. [2016] identified two types of firm engagement: community-focused firms (e.g., GNOME), which prioritize building vibrant OSS communities and monetize through services (which favor restrictive licenses so that a competitor can not create proprietary enhancements without sharing the code), and product-focused firms (e.g., Eclipse), which rely on product revenues and thus favors weaker copyleft provisions to allow some forms of bundling with proprietary code.

¹The term "nonmarket" excludes for-profit organizations.

Emergence of cloud services endangered the business model of OSS companies that rely on service contracts. Because cloud companies provide computing services, they can also run (and sell) the same OSS software as a service, thus cutting out companies that developed the software. For example, in 2021, Elasticsearch changed its Apache license to a commercial one where the products built from that code can not be provided to others as a managed service. They made another change in 2024, moving to Affero GPL, which is a copyleft license that requires anyone running a modified program on a server and letting other users communicate with it to also provide access to the source code of the modified version running on the server. As we can see, the more permissive license (Apache) was changed to a more restrictive one (commercial, then AGPL), as is typical for the service model.

Despite significant amount of research on the motivations to participate in OSS and commercial involvement, few of the key predictors can be reliably operationalized at scale. Based on the Fershtman and Gandal [2007] conjecture that restrictive licenses attract more contributors per project, we can expect that larger projects (in terms of contributor count) will adopt restrictive licenses.

Another key factor is diversity in OSS projects. Research shows that teams with greater diversity are correlated with higher levels of innovation and project quality [Vasilescu et al. 2015]. Projects with more diverse teams may favor licenses that balance openness and protection, choosing options that ensure inclusivity and equitable contributions while also allowing some flexibility in how the code is used and shared. While diversity encompasses several dimensions (e.g., ethnicity, experience), uniformly operationalizing these across multiple dimensions is not feasible due to the lack of consistent data. Therefore, we use the number of female authors as a rough proxy for team diversity in this study.

In summary, we hypothesize that:

- **Hypothesis (H1a):** Higher total numbers of authors and core authors² correlate with more restrictive licenses.
- **Hypothesis (H1b):** Higher team diversity (approximated by female author involvement) correlates with neither too restrictive nor too permissive licenses.
- **Hypothesis (H1c):** Ideologically motivated copyleft licenses will be more likely to be retained than other types of licenses.

2.2 Social and Technical Choice

Licenses, like other technologies and practices, spread through communities. Prior studies, such as those by Ma et al. [2020] and others [Lamba et al. 2020; Malviya-Thakur et al. 2023], used the theory of social contagion [Shoroye et al. 2015] to estimate the impact of exposure, susceptibility, and infectiousness on developer choices regarding package (and tool) selection. These studies found that exposure measured by the number of overall deployments at the time of choice, had a significant positive effect on package (tool) selection.

We expect a similar pattern for license selection. One dimension of exposure is the prevalence of a license (which is relatively constant over time) and favors the selection of popular licenses. i.e., developers who are indifferent or unaware of the differences between licenses are likely to gravitate toward those they have encountered. Just as developers tend to choose packages that are technically compatible with their existing technology stack and socially aligned with their collaborators' choices, we anticipate a similar behavior in license choice. Developers are likely to select a license compatible with their current projects or one already used by their collaborators (also, see "Kinship" motivation above).

²Those responsible for 80%+ of the commits

The choice of programming language also influences license selection due to the culture and practices embedded within specific developer communities. For example, languages like Python and JavaScript, commonly used in web development, often favor permissive licenses like MIT or Apache, which emphasize reuse and flexibility [Lerner and Tirole 2005]. In contrast, languages such as C or C++, which are dominant in systems programming, tend to be linked with copyleft licenses like GPL, prioritizing keeping derivative works open [Fitzgerald 2006]. The language captures the influence of community norms and technical compatibility on license selection.

The number of forks, stars, and community size associated with a project can also impact license decisions. Tsay et al. [2014] found that projects with a large number of forks are perceived as more valuable and trustworthy by the community. Projects aiming to encourage collaboration and reuse to maintain this momentum may favor permissive licenses. Similarly, Borges and Valente [2018] suggest that the number of stars signals community approval, which, like forks, may influence the choice of more permissive licenses.

Furthermore, the number of downstream projects a project interacts with can play a crucial role in license selection. If a project interacts with many downstream projects, it may opt for licenses that balance flexibility and control, favoring weaker or conditional licenses over highly permissive or restrictive ones.

The decision to count downstream projects is motivated by the need to quantify a project's external interactions. While analyzing the specific licenses of these projects would provide more detailed insights, counting them offers a practical approach to estimate the complexity and potential challenges related to license alignment. Based on this discussion, we hypothesize that:

- **Hypothesis (H2a):** The overall popularity of the license and programming language-specific popularity will favor its choice.
- **Hypothesis (H2b):** Higher numbers of forks, stars and downstream projects correlate with permissive licenses.

2.3 Legal and Regulatory Context

Open source software licenses serve not only as technical choices but also as legal instruments defining the rights and obligations of developers and users. The selection of a license carries significant implications for intellectual property, liability, and compliance with copyright laws.

Wu et al. [2024] conducted a large-scale empirical study revealing the complexities of license compliance and the frequent misunderstandings developers face, especially in large projects. Similarly, Cui et al. [2023] found that over 25% of the 16,000 open source projects they analyzed contained license conflicts, often due to misunderstandings of license terms. Their tool, DIKE, helps detect such conflicts, underscoring the challenges of managing multiple licenses within a project.

Almeida et al. [2017] further emphasized the widespread difficulty developers have in correctly interpreting licenses, which can lead to non-compliance and legal risks. These findings highlight the need for better education and tools to support effective license management.

Multiple studies have shown that declared project licenses are often inaccurate or incomplete [German et al. 2010; Reid and Mockus 2023; Wolter et al. 2023]. Wolter et al. [2023] found that nearly 50% of open source projects on GitHub exhibited inconsistencies between the declared top-level license and the actual licenses used within the codebase.

Xu et al. [2023a,b] contributed to addressing these challenges with tools like LiDetector, which detects license incompatibilities, revealing that over 70% of projects studied had such issues. Their follow-up tool, LiResolver, provides solutions for resolving these incompatibilities, demonstrating the critical role of automated tools in managing license compliance.

Regulatory frameworks, such as the European Union’s General Data Protection Regulation (GDPR) and U.S. export control laws, add further complexity to licensing decisions, particularly in international and data-sensitive projects. These regulations can influence license choices by introducing additional compliance challenges.

Delays in adopting a license often reflect the need for careful legal deliberation to ensure compliance with intellectual property laws and other legal frameworks. Therefore, we anticipate that less prevalent licenses will be chosen after longer delays.

Additionally, projects that interact with upstream components may prioritize selecting legally aligned licenses to facilitate integration [Cui et al. 2023; Xu et al. 2023b].

Furthermore, larger projects, especially those with many contributors, are more likely to face legal challenges regarding intellectual property, contributor agreements, and international regulations. In such cases, delayed license adoption may reflect careful legal evaluation aimed at mitigating risks [Fershtman and Gandal 2007].

In summary, we hypothesize that:

- **Hypothesis (H3a):** Longer delays in adoption will favor less popular licenses.
- **Hypothesis (H3b):** Projects with more contributors will favor restrictive licenses.

2.4 Project Needs

While legal compliance sets essential boundaries, the specific needs and objectives of a project further influence the selection of an appropriate license.

Developers often choose licenses based on their preferences for the future use of the software. Research by Sen et al. [2008] highlights that developers’ motivations — such as a desire for widespread adoption or strong copyleft enforcement — play a significant role in license selection.

The choice of license can significantly influence a project’s development trajectory. Colazo and Fang [2009] found that copyleft licenses promote greater permanence and long-term involvement in projects, affecting both developer contributions and engagement.

The number of commits is a key indicator of project activity and maintenance. Koch and Schneider [2002] suggests that projects with frequent commits tend to have more active development cycles, making them more reliable and attractive for reuse. As project activity increases, maintainers may select licenses that either encourage contributions or protect the evolving codebase from potential misuse, balancing the needs for stability and control.

The number of files in a project reflects its complexity and modularity. Mockus [2007] observed that larger projects with more files are more likely to offer reusable components, making license choice critical for balancing reuse and intellectual property protection. Bird et al. [2009] further suggested that projects with more files tend to have modular structures, which encourage community engagement and adoption, potentially favoring more restrictive licenses.

Time-based metrics — such as the earliest and latest commits and the number of active months — provide insights into a project’s maturity and stability. Capiluppi et al. [2003] noted that older, more mature projects are often seen as more reliable, which could influence the choice of licenses that support long-term sustainability, such as copyleft licenses. On the other hand, newer projects may opt for permissive licenses to facilitate rapid contributions during active development phases.

Burstiness, as a proxy for fluctuating activity, was operationalized by measuring the number of months in which the project had activity over its lifetime (from earliest to latest commit). These projects may not focus heavily on legal concerns and tend to choose well-known, widely used licenses as a quick, familiar option, rather than spending time evaluating more specific licensing terms.

- **Hypothesis (H4a):** More active projects in terms of number of commits will favor neither too restrictive nor too permissive licenses.
- **Hypothesis (H4b):** More complex projects approximated by the number of files will favor restrictive licenses.
- **Hypothesis (H4c):** Older projects will favor restrictive licenses that support long-term sustainability.
- **Hypothesis (H4d):** Burstiness reflects the lack of stability and favors more popular licenses.

3 Comparisons to Prior Work

Despite legal importance of license choice and the potential to stunt innovation and poison supply chains, it appears that a significant portion of OSS participants are ignoring the need to choose a license or to verify license compatibility. We construct a preliminary theory of license choice by reviewing relevant literature, operationalizing potential drivers of license selection, and testing that theory. However, while our work is not the first to empirically study software licenses, it differs from the literature in two important ways.

Comprehensive Identification of Licenses. Most prior research, such as the work by Wu et al. [2024] and Xu et al. [2023b], primarily depends on explicit license declarations found in metadata files. Others, like Feng et al. [2019], apply static analysis on binaries to detect embedded license texts. However, these methods may overlook licenses that are not clearly stated or are stored in unconventional directories. In contrast, we use a dataset by Jahanshahi et al. [2024b], which follows a more comprehensive approach by scanning virtually the entire OSS ecosystem for any files containing “license” in their filepath. Moreover, they employ the winnowing algorithm, a robust method for matching license texts to known licenses, enhancing the accuracy of detecting both partial and full matches, even when the text is embedded or slightly modified. This method captures not only standard license files but also other files potentially holding licensing details, ensuring that no relevant license information is missed.

Scale and Scope of Analysis. Previous works often limit their scope to specific platforms (e.g., GitHub), a few package manager environments (e.g., NPM), or types of licenses (e.g., OSI-approved licenses), lacking a comprehensive, large-scale approach to detecting and analyzing licenses. Our study expands this scope by analyzing essentially the entire open source landscape, ensuring a more comprehensive cross-platform understanding of licensing practices.

In Table 1, we compare our study’s methodology, coverage, scope, and scale with the most recent comparable studies in the field.

Table 1. Latest Studies in Open Source Software Licensing

	License Detection	Coverage	Scope	Scale (Projects)
[Wu et al. 2024]	Package Manager API	License Files	Maven, NPM, PyPI, RubyGems, Cargo	3,474,778
[Cui et al. 2023]	NLP/Extract License Terms	All Files	GitHub - >1000 Stars	16,341
[Wolter et al. 2023]	GitHub API+NLP/Extract License Terms	All Files	GitHub - Cataloged by OpenHub	1,000
[Xu et al. 2023b]	NLP/Extract License Terms	All Files	GitHub - Popular Python Projects	1,846
.....				
This Study	NLP/Matching Known Licenses	License Files	Entire OSS	131,171,379

4 Methodology

4.1 License Types

We group licenses based on their characteristics. This grouping helps categorize and understand the different ways software can be distributed and modified. These categories typically include

permissive, copyleft, weak copyleft, and public domain/unlicensed licenses. This classification is widely recognized in the field and is supported by various scholarly sources. As noted by Kaminski and Perry [2007], these licensing patterns have shaped the development of OSS.

1. *Permissive Licenses*: These licenses, such as the MIT and BSD licenses, are known for their minimal restrictions on how the software can be used. They allow software to be freely used, modified, and redistributed, even as part of proprietary software [Kapitsaki and Charalambous 2019]. This permissiveness promotes wider adoption and integration of the software in diverse projects, including commercial applications [Kapitsaki et al. 2022].

2. *Copyleft Licenses*: These licenses, like the GNU General Public License (GPL), require that any modified versions of the software must also be distributed under the same license. This ensures that derivative works remain free and open, thus preserving the original freedoms granted by the license [D’Antoni and Rossi 2007]. This characteristic is essential for maintaining the open source nature of software, as it prevents proprietary modifications [Laurent 2012].

3. *Weak Copyleft Licenses*: These licenses, such as the GNU Lesser General Public License (LGPL) and Mozilla Public License (MPL), strike a balance between permissive and strong copyleft licenses. They allow linking with proprietary software under certain conditions, while modifications to the licensed components themselves must remain open [Alamoudi et al. 2020]. This flexibility encourages the use of open source libraries in both open and closed source projects [Gamalielsson and Lundell 2017].

4. *Conditional Open Licenses*: These licenses include specific conditions that must be met for usage, such as attribution (CC-BY), non-commercial use, or share-alike requirements. Creative Commons licenses, like CC-BY and CC-BY-SA, provide a framework for sharing while respecting the creator’s conditions [de Rosnay 2009].

5. *Public Domain/Unlicensed*: Public domain and unlicensed software, including those using the Creative Commons Zero (CC0) license, are not restricted by copyright law. The authors of such software waive all rights, allowing anyone to use, modify, and distribute the work without any conditions. This category provides the maximum freedom for software use and is often utilized for simple, non-critical software components.

4.2 World of Code Infrastructure

World of Code (WoC)³ is an infrastructure designed to cross-reference source code change data across the entire FLOSS community, facilitating sampling, measurement, and analysis within and across software ecosystems [Ma et al. 2019, 2021]. In essence, it is a software analysis pipeline that encompasses the discovery and retrieval of data, storage and updates, and the transformations and data augmentation required for downstream analytic tasks [Ma et al. 2021].

WoC offers various maps that connect git objects and metadata (commits, blobs, authors) to each other. It also provides higher-level maps, such as project-to-data connections (e.g., project-to-author), author aliasing [Fry et al. 2020], and project deforking maps [Mockus et al. 2020]. We use the project-to-license (P2L) map [Jahanshahi et al. 2024b] from WoC that provides all the times at which a license was committed to a project and also verifies whether it still exists in the project’s latest version.⁴ We also employ the concept of deforked projects as introduced by [Mockus et al. 2020] to avoid potential biases from forks and duplicates of the same project. Throughout this paper, the term “project” refers to this deforked project unless stated otherwise.

³<https://worldofcode.org>

⁴Version V, latest at the time of this study.

4.3 Regression Model and Sample

To test our hypotheses, we selected a regression model appropriate to the nature of our response variable – specifically, multinomial logistic regression. This model is ideal because the response variable – the license type – is categorical with multiple possible outcomes, allowing us to model the probability of each license type based on project characteristics [Hosmer Jr et al. 2013].

Since the target population encompasses virtually the entirety of OSS projects and given its scale and heterogeneity, we employed a stratified sampling approach to enhance the representativeness of our sample [Thompson 2012]. We stratified projects using six key variables: the number of commits, the number of blobs, the number of authors, the number of forks, the number of active months, and the earliest commit time of the project. These variables were selected because they are important indicators of project size, activity, and historical context, which are likely to influence the outcomes of our analysis [Crowston and Howison 2005; Koch and Schneider 2002].

The WoC's MongoDB project database comprises 131 million OSS projects. A notable feature of the data is its heavy right skewness, common in large datasets with many small and inactive projects. To address this skewness and ensure meaningful strata, we binned each variable as follows: commits (fewer than 500, 500–2000, and over 2000), blobs (below or above 10,000), authors (one, 2–10, and more than 10), forks (none or at least one), and active months (less than three or more than three). These classifications reflect different levels of activity, collaboration, complexity, and community engagement.

Projects were also divided into four historical eras based on their first commit: the Foundational Era (before 1998), marking the early stages of open source; the Dot-com Boom and OSS Expansion (1998–2010), signifying rapid growth and adoption; the Maturation and Mainstream Adoption period (2010–2018), when open source became key to enterprise software; and the Modern Era (2019–present), reflecting recent trends in community engagement and cloud-native technologies.

These stratifications produced 288 unique bins ($3 \times 3 \times 2 \times 2 \times 4$). We randomly selected 5,000 projects per bin, targeting a total of 1,440,000 projects. However, due to uneven distribution, some bins contained fewer projects, resulting in a final sample of around 500,000 projects. Despite this, the large sample size and stratified approach ensure a balanced and comprehensive representation of the OSS ecosystem, supporting robust and generalizable conclusions in our regression analyses.

4.4 Modeling Considerations

To validate our model, we checked for correlations among predictors, as high correlations make results unstable, inflate standard errors, and make coefficients hard to interpret. Addressing this issue improves the model's validity [Dormann et al. 2013]. We used a 0.6 correlation threshold, as is commonly used to reduce the chances of multicollinearity while retaining key predictors. This cutoff strikes a balance, preserving some natural correlations while ensuring predictor independence [Vatcheva et al. 2016].

To improve model interpretability, we used the *emmeans* package in R to compute marginal probabilities across outcome variable levels [Lenth 2024]. This allows us to assess predicted probabilities for each license type based on programming language, while controlling for other predictors.

For numerical predictors, we estimated marginal probabilities at key quantiles (25th, 50th, and 75th), capturing variability in predicted outcomes.⁵ This method helps interpret complex models, such as multinomial logistic regression, by focusing on marginal probabilities instead of coefficients, making results clearer and more actionable [Agresti 2012] (see Figure 3). After estimating marginal probabilities, we used the *pairs* function from the *emmeans* package for pairwise comparisons to

⁵Exceptions were made for variables with skewed distributions, such as female authors and forks, where the 25th and 50th quantiles were both 0; here, we used 0, 1, 3 and 0, 2, 5, respectively. For adoption delay, we used 1, 3, and 12 months.

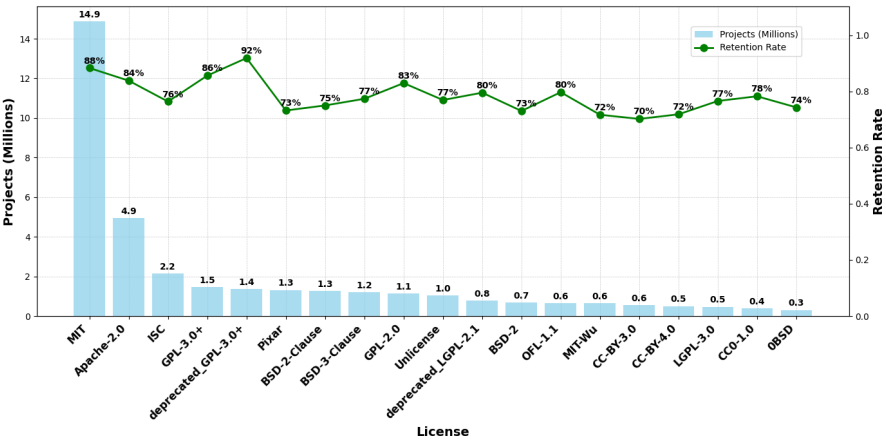


Fig. 1. The distribution and retention rates of the most used licenses.

identify significant differences between predictor levels and outcome categories. We applied the Tukey method to control for family-wise error rates [Hothorn et al. 2008; Hsu 1996].

The pairwise comparison analysis was conducted in two ways: comparing outcome probabilities within predictor levels (e.g., programming language) and comparing predictor levels within each outcome category. This bidirectional approach allows for a thorough exploration of the interaction between predictors and outcomes.

5 Results and Discussion

We organize our results in two parts, starting with a broad exploration of license types before continuing with the regression analysis.

5.1 The Types and Prevalence of Licenses Across the OSS Landscape

5.1.1 Our Findings. We first counted the total number of projects that had at least one occurrence of any license during their lifetime. We then identified the top 20 most used licenses, as shown in Figure 1. Out of the 131,171,379 projects indexed by WoC, 22,281,342 projects had at least one license committed to their project at some point in their lifetime. However, when examining the latest version of these projects, this number drops to 20,110,256 projects.

When interpreting the numbers shown in these figures, it is important to note that the sum of these numbers exceeds the total number of projects with a license. This discrepancy arises because some projects have multiple licenses and are therefore counted in several categories.

To calculate the retention rate for each license, we determined the number of projects that still had the license in their latest state and divided this number by the total number of projects that had ever used the license. This retention rate indicates the proportion of projects that continue to use the license over time, and it is also displayed in Figure 1.

As shown in the figure, the MIT license is the most prominent, followed by the Apache 2.0 and ISC licenses. Notably, per H1c, the deprecated GPL 3 license (with strong copyleft provisions) exhibits the highest retention rate. In contrast, the Creative Commons 3 license (CC-BY-3.0) shows the lowest retention rate among the top 20 most used licenses.

We also analyze the distribution of projects according to their license types. Initially, we consider the distribution across the entire project history, meaning that if a project adopted a license at

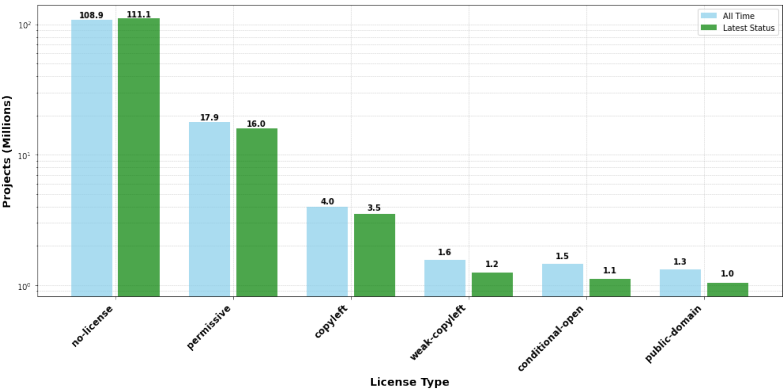


Fig. 2. The distribution of license types across projects.

Table 2. Comparison between our results and the literature.

	Projects	No License	Copyleft	Weak Copyleft	Conditional Open	Permissive	Public Domain
[Wu et al. 2024]	3,474,778	21.12%	5.72%	5.12%	-	89.16%	-
[Cui et al. 2023]	16,341	10.51%	11.32%	0.97%	-	87.71%	-
This Study	131,171,379	83.01%	15.35%	5.26%	4.82%	70.18%	4.39%

any point and later removed it, it is still counted as having that license. Next, we determine the distribution based on the most recent status of each project. Figure 2 shows that “conditional-open”, “weak-copyleft”, and “public-domain” license types have the lowest retention, with approximately a quarter of the projects that ever had such licence changing it by their latest version. For comparison, only approximately 8% of of “copyleft” licences were changed, which is consistent with our theory (H1c) that ideology-related licence choice should be most “sticky.”

5.1.2 Comparison with Literature. Table 2 provides a comparison between the results in the literature and our study, highlighting substantial differences primarily due to variations in scope, methodology, and dataset size (see Table 1).

A key methodological difference between our study and Wu et al. lies in the treatment of data points. Wu et al. considered each version of a package as a separate data point, resulting in 46.59 million data points from 3.47 million projects. This approach could introduce bias by overrepresenting projects with many versions, potentially skewing observed trends in licensing. Conversely, our study treated each project as a single data point, providing a more balanced view of the open source ecosystem without overemphasizing projects with frequent updates.

The focus of the Wu et al. study on only five package managers might explain the significant discrepancy in the percentage of projects without a license. Their study reported that 21.12% of projects lacked a license,⁶ whereas our study found a much higher percentage of 83.01%. On the other hand, Cui et al. reported that only 10.51% of projects were unlicensed. The difference with Wu et al. likely stems from the fact that the package managers they examined are associated with more mature and well-maintained projects, which are more likely to have defined licenses. Additionally,

⁶Since the paper provides these percentages by package manager, we calculated the overall percentage by taking a weighted average based on the number of projects per package manager.

Cui et al. filtered their data to include only projects with over 1,000 stars, which might explain why the percentage of unlicensed projects is so low. In contrast, our broader dataset includes a wide variety of projects, many of which may be less mature or less formally governed, leading to a higher incidence of unlicensed projects.

The analysis of copyleft versus permissive licenses further reflects the differences in scope. Wu et al.'s study found a dominance of permissive licenses, with much lower percentages for copyleft and weak copyleft licenses. However, our study, covering all programming languages, reveals a more diverse licensing landscape, with a lower percentage of permissive licenses and higher percentages of copyleft licenses. This suggests that by focusing on just five package managers, Wu et al.'s study might underrepresent broader licensing trends, particularly the prevalence of copyleft licenses across the global open source ecosystem.

Key Findings

- (1) Overall, 83% of the projects in the dataset lack a formal license, which is significantly higher than the figures reported in previous studies.
- (2) The MIT license is the most widely used license, accounting for 65% of all licensed projects and 10% of all OSS projects, supporting H2a.
- (3) Despite the overall preference for permissive licenses, copyleft licenses such as GPL exhibit strong retention rates as suggested by H1c.
- (4) The differences in license usage reported in our study compared to previous literature highlight the importance of dataset scope and methodology when assessing the prevalence of licenses in OSS projects.

5.1.3 Implications. The analysis of OSS licensing reveals that 83% of projects lack a formal license, posing significant legal risks and discouraging innovation by preventing responsible actors from reusing unlicensed code. This highlights the need for developers to prioritize licensing to set clear terms of use. Our findings challenge previous studies, which often underestimate the prevalence of unlicensed projects by focusing on mature ones, missing the broader risks of software reuse.

Even though many of these unlicensed projects may seem trivial, their public availability still creates legal ambiguity and poses a risk of license noncompliance, especially when combined with the lack of thorough legal review in many downstream projects. [Jahanshahi et al. \[2024a\]](#) demonstrated that in the context of copy-based reuse, nearly 18% of reused artifacts originated from very small projects,⁷ while large projects⁸ accounted for only 32% of reused artifacts. This highlights the importance of addressing the issue of unlicensed code within the OSS ecosystem.

Among projects that are licensed, the MIT license is the most prevalent, reflecting a historic preference for permissive licenses that facilitate broad adoption and collaboration. However, the continued use of copyleft licenses like the GPL (which was popular decades ago) demonstrates that many developers are committed to preserving the open nature of derivative works.

5.2 The Impact of Project Characteristics on License Choice

5.2.1 Our Findings. Now we turn to the results of our multinomial regression model discussed in Section 4.3. The basic statistics of the variables, including the 5th percentile, median, mean, and 95th percentile for the numeric variables, as well as the counts of different levels for the factor

⁷Projects with no stars and fewer than 10 commits.

⁸Projects with more than 10 stars and 100 commits.

Table 3. License choice model - descriptive statistics.

Variable		Description						Statistics	
License	No License	Permissive	Copyleft	Weak Copyleft	Conditional Open	Public Domain			
Counts (%)	263,974 (46.13%)	148,320 (25.92%)	60,925 (10.65%)	43,143 (7.54%)	30,933 (5.41%)	24,878 (4.35%)			
			5%	Median	Mean	95%			
EarliestCommit	Time since the earliest commit		05/08/2006	07/05/2017	01/31/2016	11/23/2021			
LatestCommit	Time since the latest commit		04/30/2011	02/17/2020	03/15/2019	04/28/2023			
CoreAuthors	Authors with 80%+ of commits		1	2	8.62	17			
FemaleAuthors	Number of female authors		0	0	2.10	6			
Forks	Number of forks		0	0	27.66	48			
Commits	Number of commits		2	155	2,982.63	5,770			
Files	Number of files		5	1,820	17,295.57	59,939.60			
AdoptDelay	Earliest commit to license adoption (days)		0	0	133	751			
DownProjects	Number of downstream projects		0	12	1,831.07	2,342			
Burstiness	(Latest - Earliest) / Active months		0	1	1.87	6.37			
Language	JavaScript	C/C++	Python	Java	PHP	Ruby	(Remaining)		
Counts (%)	221,588 (38.72%)	82,551 (14.43%)	53,468 (9.34%)	50,372 (8.80%)	44,952 (7.86%)	18,592 (3.25%)	100,650 (17.59%)		

variables, are presented in Table 3. Given that the response variable has multiple outcome levels, the model provides a distinct set of coefficients for each outcome level relative to a chosen base level. This approach is typical in models such as multinomial logistic regression, where the base level acts as a reference category for comparing all other outcome levels. Each set of coefficients indicates the influence of the predictor variables on the probability of the response variable assuming a specific outcome level relative to the base level. In our model, the base level is defined as projects with “no license,” so the coefficients reflect the impact of the predictors on each license type compared to having no license. It is important to note that the selection of the base level is arbitrary and does not affect the estimated probabilities of each outcome level, as the relative probabilities between outcome levels remain constant regardless of the chosen base level [Agresti 2012].

The ANOVA results (see Table 4) for the fitted model shows that the p-values for all the predictors are close to zero, indicating that each predictor significantly improves our model’s fit.

Table 4. License choice model - ANOVA table

Variable	DF	Deviance	Resid. DF	Resid. Deviance	p.value
NULL			572,168	1,641,371.30	
EarliestCommit	5	22,631.53	572,163	1,618,739.77	<0.01
LatestCommit	5	15,003.28	572,158	1,603,736.49	<0.01
CoreAuthors	5	4,106.14	572,153	1,599,630.35	<0.01
FemaleAuthors	5	766.09	572,148	1,598,864.26	<0.01
Forks	5	10,115.52	572,143	1,588,748.74	<0.01
Commits	5	9,304.28	572,138	1,579,444.46	<0.01
Files	5	173,992.60	572,133	1,405,451.86	<0.01
Delay	5	134,793.54	572,128	1,270,658.31	<0.01
DownProjects	5	7,137.78	572,123	1,263,520.53	<0.01
Burstiness	5	1,284.30	572,118	1,262,236.24	<0.01
Language	55	57,669.49	572,063	1,204,566.75	<0.01

Our findings offer mixed support for the hypothesis. As the number of core authors increases, the probability of adopting a permissive licenses increase (56.91% to 57.91%).

This contradicts H1a that projects with restrictive licenses tend to have more contributors.

In terms of diversity (approximated by the number of female authors), the likelihood of adopting conditional open and public domain licenses increases slightly, while copyleft licenses show

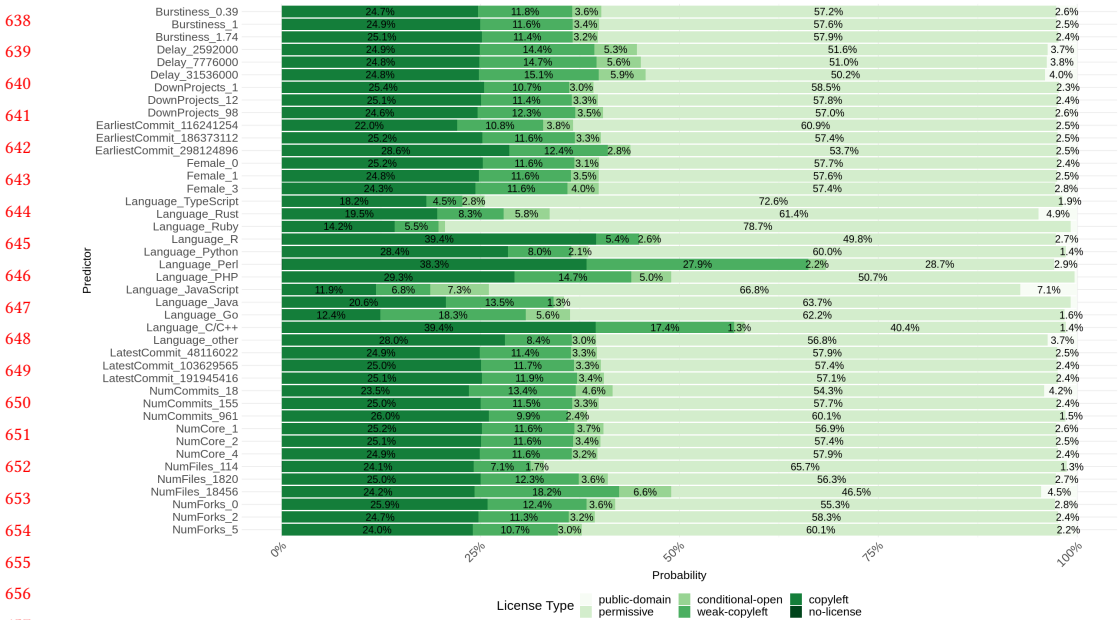


Fig. 3. License choice model - estimated marginal probabilities

modest declines, suggesting a preference for flexible but protective licenses in more diverse teams, supporting the diversity-related aspect of our hypothesis H1b.

Our analysis shows significant variation in licensing preferences across programming languages, reflecting the community culture and technical norms associated with each language (as predicted by H2a). JavaScript, Ruby, TypeScript, and Go exhibit a strong preference for permissive licenses (e.g., MIT, Apache), likely due to their dominant role in web development and collaborative environments or because these communities started from using such licenses. Conversely, Perl, R, and C/C++ favor copyleft licenses (e.g., GPL), aligning with their historical ties to the early days of open-source movement when copy-left licenses were more common (H2a) (see Figure 4).

As the number of forks increases, projects show a clear shift toward permissive licenses, with the likelihood rising from 55.26% to 60.10%, while the adoption of copyleft and conditional licenses declines slightly. This supports the H2b that projects with more forks, perceived as valuable by the community, tend to prioritize collaboration and reuse by favoring permissive licenses.

For downstream projects, the likelihood of selecting conditional open, public domain, and weak copyleft licenses increases modestly. Meanwhile, both permissive and strong copyleft licenses see a slight decline. This suggests that developers with more downstream interactions are leaning toward less restrictive licenses, supporting H2b. This is reflected by the decrease in strong copyleft and increase in weak copyleft, possibly driven by motivations similar to Hypothesis H1c. Additionally, the increase in public domain licensing and decline in permissive licenses may also reflect a shift toward even fewer restrictions.

As the delay in adopting a license increases (from 1 to 12 months), projects show a slight increase in the likelihood of selecting conditional open, public domain, and weak copyleft licenses. Meanwhile, the probability of choosing a permissive license decreases. This suggests that longer delays allow more time for legal and community considerations, leaning towards less known licenses. The probability of selecting a copyleft license remains stable, suggesting it is less influenced by

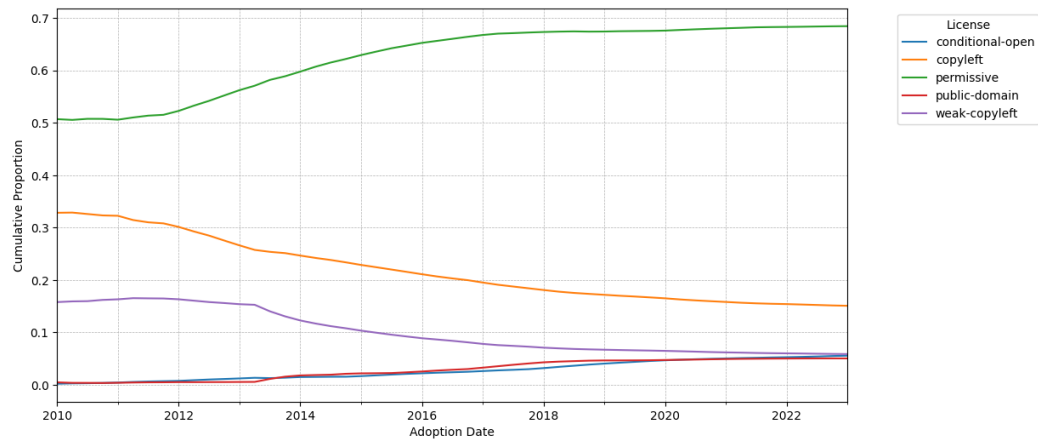


Fig. 4. Proportion of license type over time

the timing of the decision. Overall, our hypothesis H3a that more delay translates in less common license is confirmed, as the permissive and copyleft licenses are the most used (known) licenses in OSS.

As project activity increases, reflected by the number of commits, there is a rise in the adoption of copyleft and permissive licenses, with the probability of conditional open and public domain licenses decreasing. This suggests that projects with more frequent development cycles may lean toward either protection via copyleft or flexibility via permissive licenses. While this partially supports H4a, that too permissive licenses (e.g. public domain) will not be favored, the increase in strong copyleft contradicts our hypothesis.

For larger projects, as measured by the number of files, there is a notable increase in weak copyleft licenses, while permissive licenses see a sharp decrease. This suggests that as complexity grows, developers may prefer licenses that offer more control and protection, aligning with the hypothesis that larger, modular projects balance reuse with intellectual property concerns, supporting H4b. In contrast, conditional open, public domain, and strong copyleft licenses show only modest increases.

Older projects, as indicated by the time since the earliest commit, show a preference for copyleft licenses, while newer projects are more likely to adopt permissive or conditional open licenses (H4c). This reflects the hypothesis that mature projects may seek sustainability and protection, while newer projects prioritize rapid contributions or, that older projects, at their start, were relatively more exposed to copyleft licenses (H2a, see Figure 4).

Inactive projects, marked by longer time since the latest commit, show slight increases in conditional open, copyleft, and weak copyleft licenses, while permissive and public domain licenses decrease. This suggests that flexible licenses may correlate with continued activity, while more restrictive licenses become prevalent as projects age. This could also be the result of older projects leaning toward more restrictive licenses.

In bursty projects (those with fluctuating activity), both permissive and copyleft licenses increase slightly, reflecting the dual need for flexibility and protection. This aligns with the hypothesis that bursty projects may prefer licenses that are more popular and used in many other projects (H4d).

5.2.2 Comparison with Literature. Table 5 provides a comparison between the results in the literature and our study, highlighting substantial differences primarily due to variations in scope and sampling strategy.

Table 5. Comparison with literature

Language	Study	Mode	Projects	Copyleft	Weak Copyleft	Conditional Open	Permissive	Public Domain
Java	[Wu et al. 2024]	Observed	513,939	5.00%	21.25%	-	73.75%	-
	This Study	Observed	50,372	22.25%	18.01%	1.84%	56.79%	1.11%
		Model		20.59%	13.53%	1.33%	63.66%	0.84%
JavaScript	[Wu et al. 2024]	Observed	2,182,959	3.75%	1.88%	-	94.38%	-
	This Study	Observed	221,588	9.15%	9.21%	16.86%	49.61%	15.18%
		Model		11.86%	6.82%	7.33%	66.83%	7.14%
Python	[Wu et al. 2024]	Observed	485,223	15.31%	4.38%	-	80.31%	-
	This Study	Observed	53,468	26.82%	9.31%	3.59%	58.10%	2.18%
		Model		28.45%	7.97%	2.11%	60.04%	1.41%
Ruby	[Wu et al. 2024]	Observed	188,807	6.25%	1.88%	-	91.88%	-
	This Study	Observed	18,592	16.22%	6.56%	0.95%	75.29%	0.98%
		Model		14.2%	5.48%	0.82%	78.67%	0.82%
Rust	[Wu et al. 2024]	Observed	103,850	5.00%	2.81%	-	92.19%	-
	This Study	Observed	3,080	17.62%	6.71%	5.4%	66.7%	3.57%
		Model		19.53%	8.34%	5.8%	61.43%	4.88%

The comparison between our study and Wu et al.’s findings highlights significant differences in licensing patterns across programming languages, emphasizing the need to control for influencing factors. Wu et al.’s analysis, based on raw data, suggests a strong preference for permissive licenses in languages like Java (73.75%), JavaScript (94.38%), and Python (80.31%). However, our study, which incorporates both observed data and a regression model controlling for time, commits, and project activity, shows lower permissive license rates (Java: 56.79%, JavaScript: 49.61%, Python: 58.10%) and a higher prevalence of copyleft and weak copyleft licenses.

Our regression model further adjusted these figures, often raising permissive license rates but still reflecting a more diverse licensing landscape, suggesting that copyleft licenses play a more prominent role than Wu et al.’s raw data indicates. The difference in findings may stem from methodology: Wu et al.’s reliance on package manager data could introduce biases toward more popular or actively managed packages, overrepresenting permissive licenses. In contrast, our stratified sampling and multinomial regression provide a broader view, revealing a more varied distribution of licenses across languages like Java, Python, and Ruby, where copyleft licenses are more common.

This comparison underscores the importance of controlling for various factors to avoid misleading conclusions, offering a more accurate understanding of license adoption in open source projects.

5.2.3 Implications. Programming language is a choice of technology that has its own libraries and tools. User of that entire ecosystem, is nudged toward the most common licences used within it. For example, C-based projects often prefer copyleft licenses, while Go and TypeScript projects lean toward permissive or conditional open licenses. Developers should be aware of how their language community influences licensing norms, and the community could offer language-specific guidelines to align new projects with best licensing practices.

The positive correlation between permissive licenses and forks suggests that developers are more likely to fork permissively licensed code. However, project maintainers have no control over the bundling of such code with proprietary software. Since more forks typically lead to greater community engagement, maintainers may need to reassess their licensing strategies to ensure

Key Findings

- (1) More core authors slightly increase permissive license adoption, while decreasing the likelihood of conditional open, and public domain licenses (H1a, H3b).
- (2) More female authors slightly increase the adoption of conditional open and public domain licenses, with a modest decline in copyleft licenses (H1b).
- (3) Permissive licenses are common in JavaScript, Ruby, TypeScript, and Go, while copyleft licenses are more prevalent in Perl, R, and C/C++ (H2a).
- (4) More forks increase the likelihood of permissive licenses, while decreasing the probabilities of conditional open, copyleft, public domain, and weak copyleft licenses (H2b).
- (5) More downstream projects slightly increase the likelihood of adopting conditional open, public domain, and weak copyleft licenses, while decreasing permissive and copyleft licenses (H2b).
- (6) Longer license adoption delays slightly increase conditional open, public domain, and weak copyleft licenses, with a slight decrease in permissive licenses (H3a).
- (7) More commits increase the likelihood of adopting copyleft and permissive licenses, while decreasing the likelihood of conditional open, public domain, and weak copyleft licenses (H4a).
- (8) As file count increases, projects are more likely to adopt weak copyleft licenses, with a decrease in permissive licenses (H4b).
- (9) Older projects favor copyleft and weak copyleft licenses, with a decrease in conditional open and permissive licenses (H4c).
- (10) Longer inactivity increases the likelihood of conditional open, copyleft, and weak copyleft licenses, while decreasing permissive and public domain licenses (H4c).
- (11) Increased burstiness slightly favors permissive licenses, with minimal changes in conditional open and copyleft licenses (H4d).
- (12) Controlling for influencing factors reveals that copyleft and weak copyleft licenses play a more significant role than previously indicated by raw data, challenging the dominance of permissive licenses reported by literature.

they align with their long-term goals. If maintainers value open collaboration but wish to prevent proprietary use, they might consider adopting licenses that offer more control, such as weak copyleft, to strike a balance between encouraging forks and protecting the project’s openness.

As the number of downstream projects increases, developers tend to favor licenses that offer more control, such as conditional open and weak copyleft licenses, reflecting a desire to retain attribution when code is copied or modified, to prevent closed-source bundling, resale as a service, or other undesired uses. The decline in permissive and copyleft licenses suggests that developers are moving away from licenses that are either too flexible or too restrictive. To address these trends, the OSS community should provide guidance and educational resources to help developers navigate these licensing decisions, particularly as downstream usage grows.

For larger, more complex projects, developers increasingly choose licenses that offer greater control, gravitating toward weak copyleft licenses while moving away from permissive ones. This shift likely reflects a need to protect the project as complexity and the risk of misuse rise. The decline in permissive licenses suggests they may be seen as inadequate for managing large, complex codebases. However, the stability of copyleft licenses shows that developers with strong ideological commitments to software freedom continue to favor these licenses regardless of project size. To support developers, the OSS community could promote license management tools and contribution guidelines that help maintain control while fostering collaboration in larger projects.

Lastly, older projects tend to favor copyleft licenses, reflecting the early OSS movement’s emphasis on preserving software freedom, while newer projects increasingly adopt permissive licenses,

prioritizing flexibility and broad adoption. This shift suggests that modern OSS development values ease of use and integration over strict control of derivative works. The community should aim to balance historical ideals of software freedom with the demand for flexibility, helping developers make informed choices that align with project goals and values.

By addressing these insights and implications, the OSS community can foster a more legally secure, inclusive, and sustainable environment for open source development, encouraging developers to make informed and strategic decisions about licensing that align with both their project's needs and the broader community's values.

6 Limitations

6.1 Project to License Map

The license map (P2L) [Jahanshahi et al. 2024b] used for our study relies on detecting license files committed to a project's repository. While this approach does not assume that licenses are consistently and accurately recorded in dedicated license.md files, it does not check the content of all files [Jahanshahi et al. 2024b]. In practice, licenses might be specified within individual source files, potentially leading to underreporting or misclassification of a project's licensing status.

Additionally, while the P2L map captures license changes over time, it may not fully account for licenses that were temporarily removed or altered before being reinstated, which could introduce inaccuracies in assessing a project's long-term licensing practices. The reliance on the latest status of licenses might also obscure important historical context, such as changes in licensing throughout a project's lifecycle, which could be crucial for understanding the project's evolution and the factors influencing license choices.

Despite the efforts of manual verification employed in P2L map, these limitations underscore the need for caution when interpreting results based on it. Complementary methods, such as cross-referencing with other data sources, may be necessary to obtain a more complete and accurate picture of licensing practices in open source software projects.

7 Conclusions

The study offers a discussion of theoretical concepts combined with rigorous statistical data analysis of the factors influencing license choices in open source software projects. The developer motivation, commercial involvement, choice theory and legal considerations were used to pose four sets of hypotheses that were evaluated on a large corpus of project/license pairs using multinomial regression modeling. Not all predictions were borne out, suggesting that prior work used to support the hypothesised effect was either no longer relevant or obtained in a different context. Key finding that over 80% of the projects have no license making their code legally virtually unusable highlight both risks of inadvertent poisoning of the software supply chain and, perhaps more importantly, the obstacle for innovation through its wider use. Many of the theorized predictions were supported by the analysis, suggesting that a more mature theory of license choice could be developed. The observed relationships may help develop recommenders that can provide projects with the most suitable default license and suggest changes as project matures or its goals change.

8 Data Availability

The replication package, including used datasets and R code are available at: [Replication Package](#)⁹

⁹<https://zenodo.org/records/13754415>

References

- Alan Agresti. 2012. *Categorical data analysis*. Vol. 792. John Wiley & Sons.
- Emad Alamoudi, Rashid Mehmood, Wajdi Aljudaibi, Aiiad Albeshri, and Syed Hamid Hasan. 2020. Open source and open data licenses in the smart infrastructure era: Review and license selection frameworks. *Smart Infrastructure and Applications: Foundations for Smarter Cities and Societies* (2020), 537–559.
- Daniel A Almeida, Gail C Murphy, Greg Wilson, and Mike Hoyer. 2017. Do software developers understand open source licenses?. In *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*. IEEE, 1–11.
- Christian Bird, Nachiappan Nagappan, Harald Gall, Brendan Murphy, and Premkumar Devanbu. 2009. Putting it all together: Using socio-technical networks to predict failures. In *2009 20th International Symposium on Software Reliability Engineering*. IEEE, 109–119.
- Hudson Borges and Marco Tulio Valente. 2018. What's in a github star? understanding repository starring practices in a social coding platform. *Journal of Systems and Software* 146 (2018), 112–129.
- Andrea Capiluppi, Patricia Lago, and Maurizio Morisio. 2003. Characteristics of open source projects. In *Seventh European Conference on Software Maintenance and Reengineering, 2003. Proceedings*. IEEE, 317–327.
- Md Atique Reza Chowdhury, Rabe Abdalkareem, Emad Shihab, and Bram Adams. 2021. On the untriviality of trivial packages: An empirical study of npm javascript packages. *IEEE Transactions on Software Engineering* 48, 8 (2021), 2695–2708.
- Jorge Colazo and Yulin Fang. 2009. Impact of license choice on open source software development activity. *Journal of the American Society for Information Science and Technology* 60, 5 (2009), 997–1011.
- Kevin Crowston and James Howison. 2005. The social structure of free and open source software development.
- Xing Cui, Jingzheng Wu, Yanjun Wu, Xu Wang, Tianyue Luo, Sheng Qu, Xiang Ling, and Mutian Yang. 2023. An empirical study of license conflict in free and open source software. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 495–505.
- Laura A. Dabbish, H. Colleen Stuart, Jason Tsay, and James D. Herbsleb. 2012. Social coding in GitHub: transparency and collaboration in an open software repository. In *CSCW*. 1277–1286.
- Linus Dahlander and Mats Magnusson. 2008. How do firms make use of open source communities? *Long range planning* 41, 6 (2008), 629–649.
- Melanie Dulong de Rosnay. 2009. Creative commons licenses legal pitfalls: Incompatibilities and solutions.
- Carsten F Dormann, Jane Elith, Sven Bacher, Carsten Buchmann, Gudrun Carl, Gabriel Carré, Jaime R García Marquéz, Bernd Gruber, Bruno Lafourcade, Pedro J Leitão, et al. 2013. Collinearity: a review of methods to deal with it and a simulation study evaluating their performance. *Ecography* 36, 1 (2013), 27–46.
- Massimo D'Antoni and Maria Alessandra Rossi. 2007. COPYLEFT LICENCING AND SOFTWARE DEVELOPMENT.
- Muyue Feng, Weixuan Mao, Zimu Yuan, Yang Xiao, Gu Ban, Wei Wang, Shiyang Wang, Qian Tang, Jiahuan Xu, He Su, et al. 2019. Open-source license violations of binary software at large scale. In *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 564–568.
- Chaim Fershtman and Neil Gandal. 2007. Open source software: Motivation and restrictive licensing. *International Economics and Economic Policy* 4 (2007), 209–225.
- Brian Fitzgerald. 2006. The transformation of open source software. *MIS quarterly* (2006), 587–598.
- Tanner Fry, Tapajit Dey, Andrey Karnauch, and Audris Mockus. 2020. A dataset and an approach for identity resolution of 38 million author ids extracted from 2b git commits. In *Proceedings of the 17th international conference on mining software repositories*. 518–522.
- Jonas Gamalielsson and Björn Lundell. 2017. On licensing and other conditions for contributing to widely used open source projects: an exploratory analysis. In *Proceedings of the 13th International Symposium on Open Collaboration*. 1–14.
- Daniel M German, Massimiliano Di Penta, and Julius Davies. 2010. Understanding and auditing the licensing of open source software distributions. In *2010 IEEE 18th International Conference on Program Comprehension*. IEEE, 84–93.
- Joachim Henkel Harhoff, Dietmar and Eric von Hippel. 2002. Profiting from voluntary information spillovers: How users benefit from freely revealing their innovations. *MIT Sloan School of Management Working Paper 4125* (May 2002).
- David W Hosmer Jr, Stanley Lemeshow, and Rodney X Sturdivant. 2013. *Applied logistic regression*. John Wiley & Sons.
- Torsten Hothorn, Frank Bretz, and Peter Westfall. 2008. Simultaneous inference in general parametric models. *Biometrical Journal: Journal of Mathematical Methods in Biosciences* 50, 3 (2008), 346–363.
- Jason Hsu. 1996. *Multiple comparisons: theory and methods*. CRC Press.
- Mahmoud Jahanshahi, David Reid, Adam McDaniel, and Audris Mockus. 2024b. OSS License Identification at Scale: A Comprehensive Dataset Using World of Code. *arXiv preprint arXiv:2409.04824* (2024).
- Mahmoud Jahanshahi, David Reid, and Audris Mockus. 2024a. Beyond Dependencies: The Role of Copy-Based Reuse in Open Source Software Development. *arXiv preprint arXiv:2409.04830* (2024).
- Halina Kaminski and Mark Perry. 2007. Open source software licensing patterns. *Computer Science Publications* (2007), 10.

- Georgia M Kapitsaki and Georgia Charalambous. 2019. Modeling and recommending open source licenses with findOSSLicense. *IEEE Transactions on Software Engineering* 47, 5 (2019), 919–935.
- Georgia M Kapitsaki, Nikolaos D Tselikas, Kyriakos-Ioannis D Kyriakou, and Maria Papoutsoglou. 2022. Help me with this: A categorization of open source software problems. *Information and Software Technology* 152 (2022), 107034.
- Stefan Koch and Georg Schneider. 2002. Effort, co-operation and co-ordination in an open source software project: GNOME. *Information Systems Journal* 12, 1 (2002), 27–42.
- Hemank Lamba, Asher Trockman, Daniel Armanios, Christian Kästner, Heather Miller, and Bogdan Vasilescu. 2020. Heard it through the Gitvine: an empirical study of tool diffusion across the npm ecosystem. In *International Conference on the Foundations of Software Engineering (FSE)*. 505–517.
- Philippe Laurent. 2012. Free and Open Source Software Licensing: A reference for the reconstruction of "virtual commons"? In *Conference for the 30th Anniversary of the CRID*. 1–19.
- Russell V. Lenth. 2024. *emmeans: Estimated Marginal Means, aka Least-Squares Means*. <https://rvlenth.github.io/emmeans/> R package version 1.10.4, <https://rvlenth.github.io/emmeans/>.
- Josh Lerner and Jean Tirole. 2002. Some Simple Economics of Open Source. *The Journal of Industrial Economics* 50, 2 (Jun 2002), 197–234. <http://www.jstor.org/stable/3569837>
- Josh Lerner and Jean Tirole. 2005. The economics of technology sharing: Open source and beyond. *Journal of Economic Perspectives* 19, 2 (2005), 99–120.
- Yuxing Ma, Chris Bogart, Sadika Amreen, Russell Zaretski, and Audris Mockus. 2019. World of code: an infrastructure for mining the universe of open source VCS data. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 143–154.
- Yuxing Ma, Tapajit Dey, Chris Bogart, Sadika Amreen, Marat Valiev, Adam Tutko, David Kennard, Russell Zaretski, and Audris Mockus. 2021. World of code: enabling a research workflow for mining and analyzing the universe of open source VCS data. *Empirical Software Engineering* 26 (2021), 1–42.
- Yuxing Ma, Audris Mockus, Russell Zaretski, Bogdan Bichescu, and Randy Bradley. 2020. A Methodology for Analyzing Uptake of Software Technologies Among Developers. *IEEE Transactions on Software Engineering* (2020). papers/TSE_choice_model.pdf
- addi Malviya-Thakur, Audris Mockus, Russel Zaretski, Bogdan Bichescu, and Randy Bradley. 2023. How R Developers explain their Package Choice: A Survey. In *International Conference on Empirical Software Engineering*. <papers/esem23survey.pdf>
- Audris Mockus. 2007. Large-scale code reuse in open source software. In *First International Workshop on Emerging Trends in FLOSS Research and Development (FLOSS'07: ICSE Workshops 2007)*. IEEE, 7–7.
- Audris Mockus, Diomidis Spinellis, Zoe Kotti, and Gabriel John Dusing. 2020. A complete set of related git repositories identified via community detection approaches based on shared commits. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 513–517.
- David Reid and Audris Mockus. 2023. Applying the Universal Version History Concept to Help De-Risk Copy-Based Code Reuse. In *2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 1–12.
- Ravi Sen, Chandrasekar Subramaniam, and Matthew L Nelson. 2008. Determinants of the choice of open source software license. *Journal of Management Information Systems* 25, 3 (2008), 207–240.
- Sonali K. Shah. 2006. Motivation, Governance, and the Viability of Hybrid Forms in Open Source Software Development. *Management Science* 52, 7 (July 2006), 1000–1014.
- Zakariyah Shoroye, Waheeb Yaqub, Azhar Ahmed Mohammed, Zeyar Aung, and Davor Svetinovic. 2015. Exploring social contagion in open-source communities by mining software repositories. In *International Conference on Neural Information Processing*. Springer, 120–127.
- Katherine J Stewart, Anthony P Ammeter, and Likoebe M Maruping. 2006. Impacts of license choice and organizational sponsorship on user interest and development activity in open source software projects. *Information Systems Research* 17, 2 (2006), 126–144.
- Steven K Thompson. 2012. *Sampling*. Vol. 755. John Wiley & Sons.
- Jason Tsay, Laura Dabbish, and James Herbsleb. 2014. Influence of social and technical factors for evaluating contribution in GitHub. In *Proceedings of the 36th international conference on Software engineering*. 356–366.
- Bogdan Vasilescu, Daryl Posnett, Baishakhi Ray, Mark GJ van den Brand, Alexander Serebrenik, Premkumar Devanbu, and Vladimir Filkov. 2015. Gender and tenure diversity in GitHub teams. In *Proceedings of the 33rd annual ACM conference on human factors in computing systems*. 3789–3798.
- Kristina P Vatcheva, MinJae Lee, Joseph B McCormick, and Mohammad H Rahbar. 2016. Multicollinearity in regression analyses conducted in epidemiologic studies. *Epidemiology (Sunnyvale, Calif)* 6, 2 (2016).
- Georg Von Krogh, Stefan Haefliger, Sebastian Spaeth, and Martin W Wallin. 2012. Carrots and rainbows: Motivation and social practice in open source software development. *MIS quarterly* (2012), 649–676.
- P Wagstrom, JD Herbsleb, RE Kraut, and A Mockus. 2010. The impact of commercial organizations on volunteer participation in an online community. In *Academy of Management Annual Meeting*.

Thomas Wolter, Ann Barcomb, Dirk Riehle, and Nikolay Harutyunyan. 2023. Open source license inconsistencies on github. *ACM Transactions on Software Engineering and Methodology* 32, 5 (2023), 1–23.

Jiaqi Wu, Lingfeng Bao, Xiaohu Yang, Xin Xia, and Xing Hu. 2024. A Large-Scale Empirical Study of Open Source License Usage: Practices and Challenges. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*. IEEE, 595–606.

Sihan Xu, Ya Gao, Lingling Fan, Linyu Li, Xiangrui Cai, and Zheli Liu. 2023a. LiResolver: License Incompatibility Resolution for Open Source Software. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 652–663.

Sihan Xu, Ya Gao, Lingling Fan, Zheli Liu, Yang Liu, and Hua Ji. 2023b. Lidetector: License incompatibility detection for open source software. *ACM Transactions on Software Engineering and Methodology* 32, 1 (2023), 1–28.

Weiwei Xu, Kai Gao, Hao He, and Minghui Zhou. 2024. A First Look at License Compliance Capability of LLMs in Code Generation. *arXiv preprint arXiv:2408.02487* (2024).

Zhiyuan Yu, Yuhao Wu, Ning Zhang, Chenguang Wang, Yevgeniy Vorobeychik, and Chaowei Xiao. 2023. Codeipprompt: Intellectual property infringement assessment of code language models. In *International Conference on Machine Learning*. PMLR, 40373–40389.

Minghui Zhou, Audris Mockus, Xiujuan Ma, Lu Zhang, and Hong Mei. 2016. Inflow and retention in oss communities with commercial involvement: A case study of three hybrid projects. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 25, 2 (2016), 13. <https://dl.acm.org/authorize?N14201>